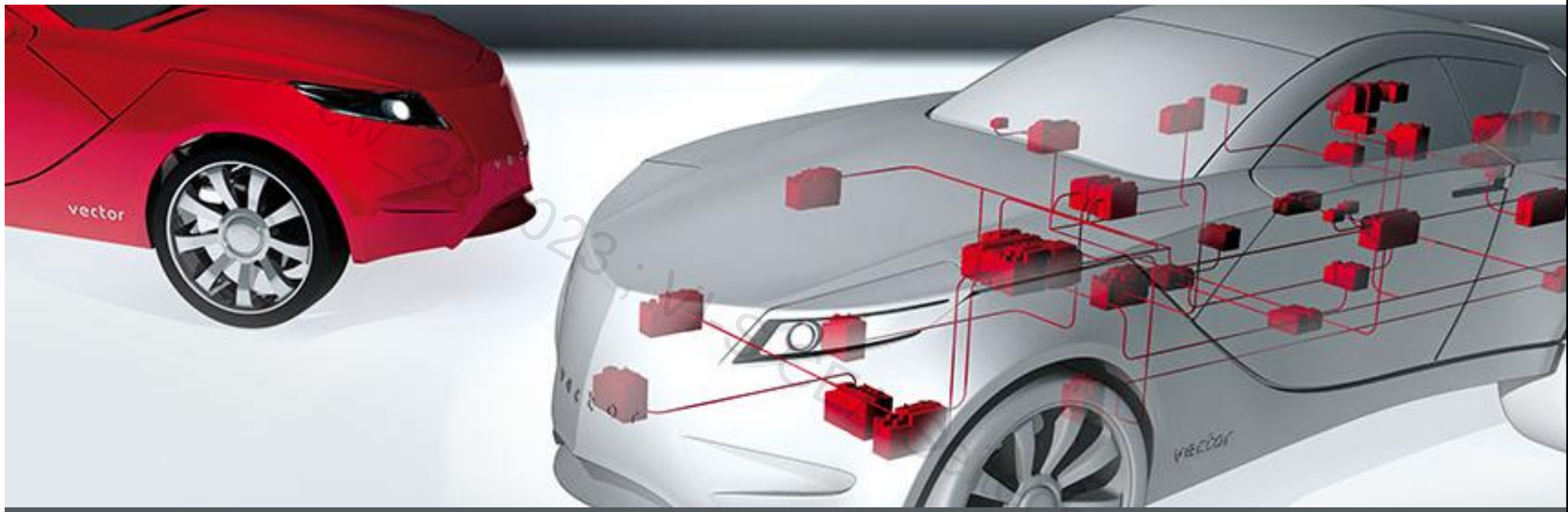
VECTOR



AUTOSAR in Practice

Operating System

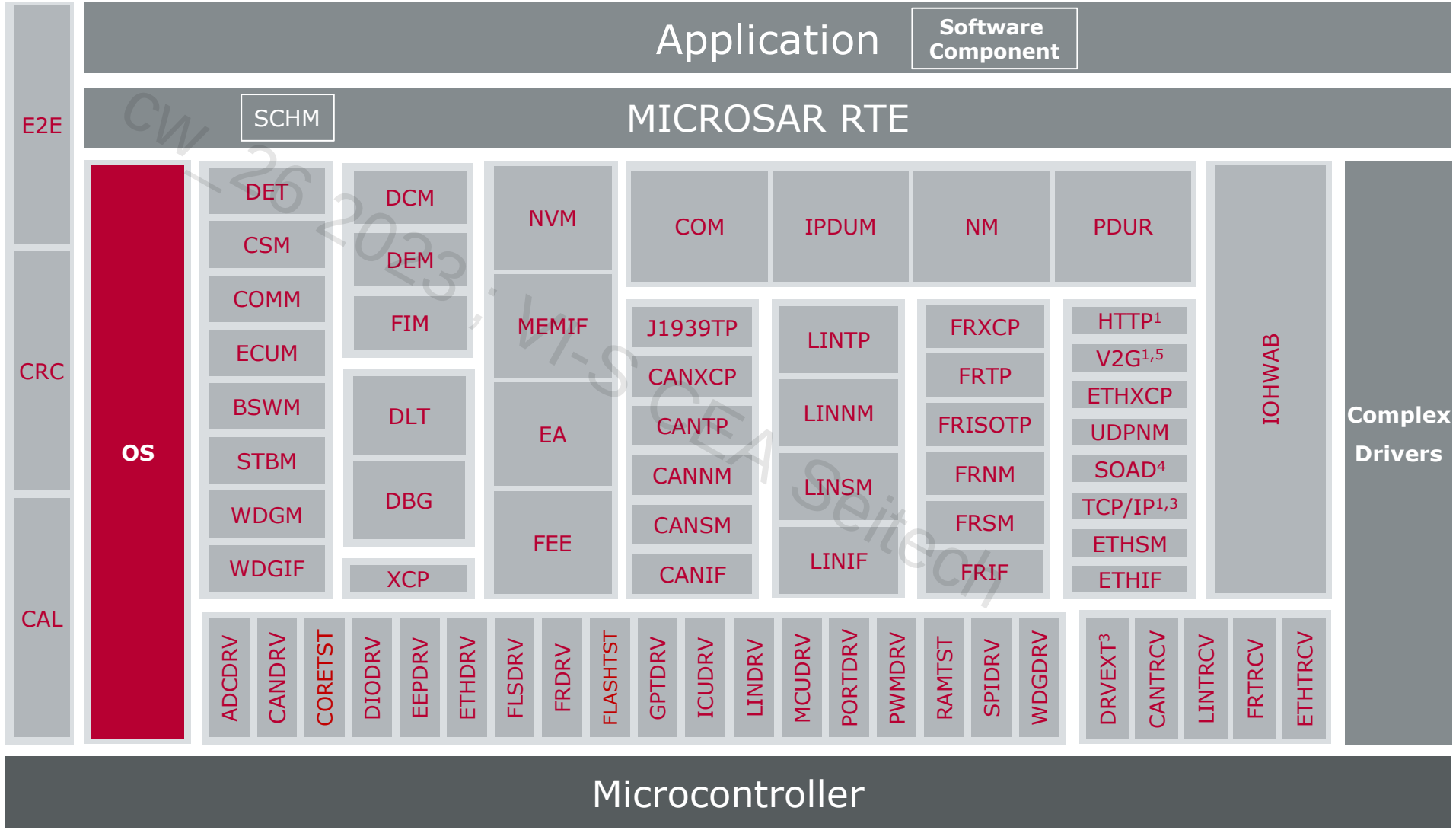
Agenda

- **Overview**
- Tasks
- Alarms
- OS counter
- Interrupts
- IOCs
- OS Resources
- Illustration
- Scalability of the OS
- Tool example
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

MICROSAR OS

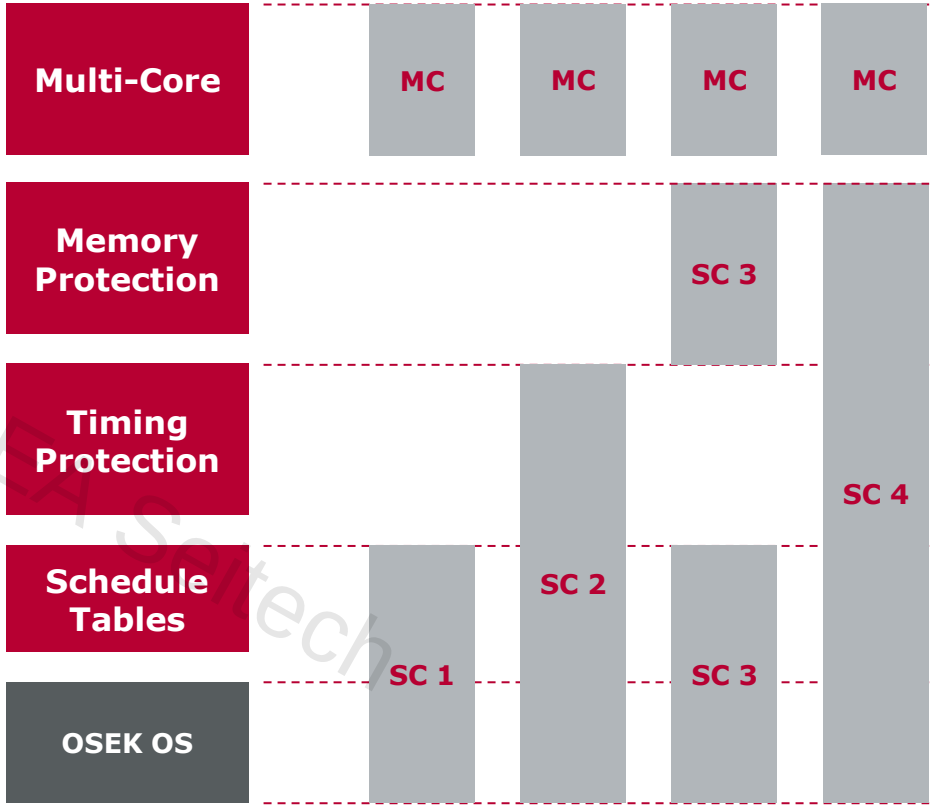
- Static operating system
- Fully-preemptive, event triggered
- Scalable in functionality and size
- Elements
  - Tasks
  - Events
  - Alarms
  - Interrupts
  - IOCs
  - Resources



CW 26 2023 - VI-SCEA Seitech

New functionality in Scalability Classes

- AUTOSAR **extends** the OSEK/VDX Operating System standard
  - AUTOSAR Workflow supported (configuration not any more over OSEK Implementation Language)
  - Additional functionality
- AUTOSAR OS add-ons are segmented into Scalability Classes (SCs)
  - Exception
    - Multi-Core support is independent of the used Scalability Class



Memory Protection

The hardware must contain a Memory Protection Unit (MPU) to support this feature.

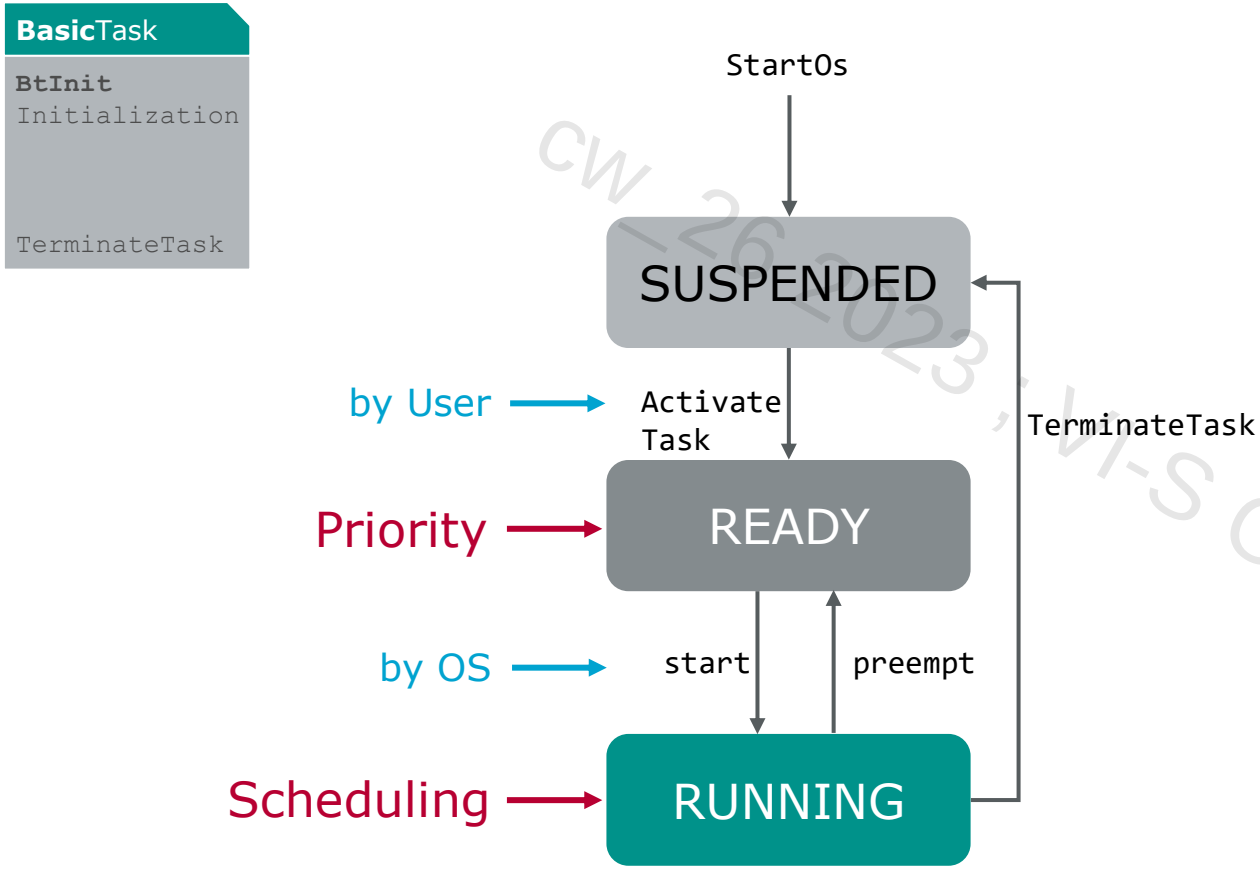
_26 2023 ; VI-S CEX Sattech

Agenda

- Overview
- **Tasks**
- Alarms
- OS counter
- Interrupts
- IOCs
- OS Resources
- Illustration
- Scalability of the OS
- Tool example
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

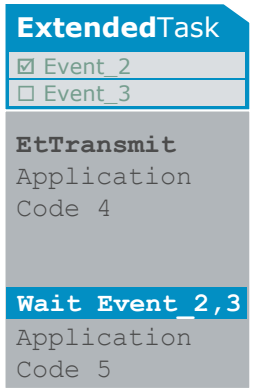
Basic Tasks

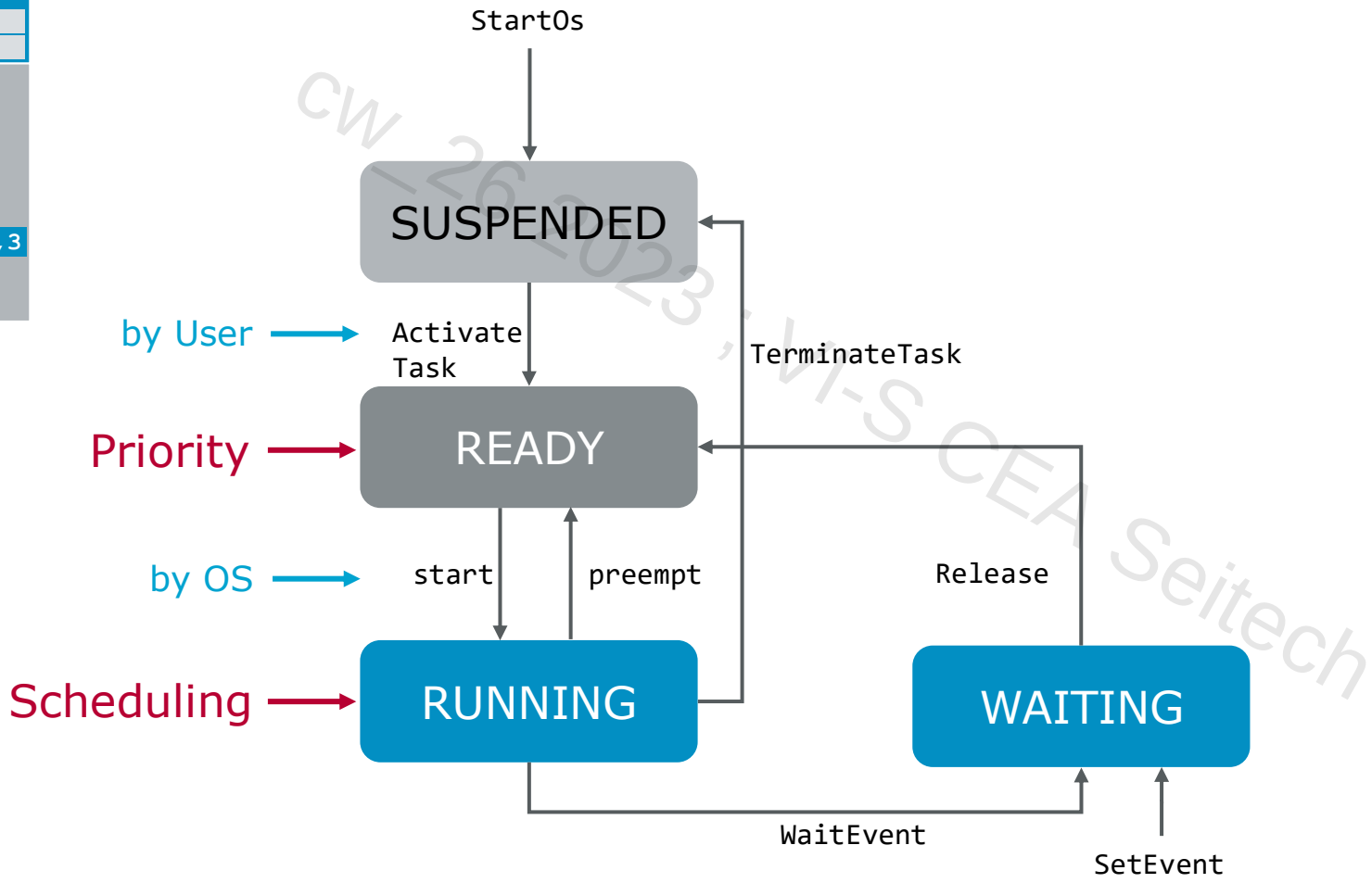


cw_26_2023 ; VI-S CEA Seitech

A **Basic Task** can be activated, runs, and terminates when finished.

Extended Tasks and Events





cw 26 2023 ; VI-S CEA Seitech

An **Extended Task** performs a blocking wait for the occurrence of certain events.

During the wait for events, the OS will schedule the next ready Task.

Events are assigned to Extended Tasks.

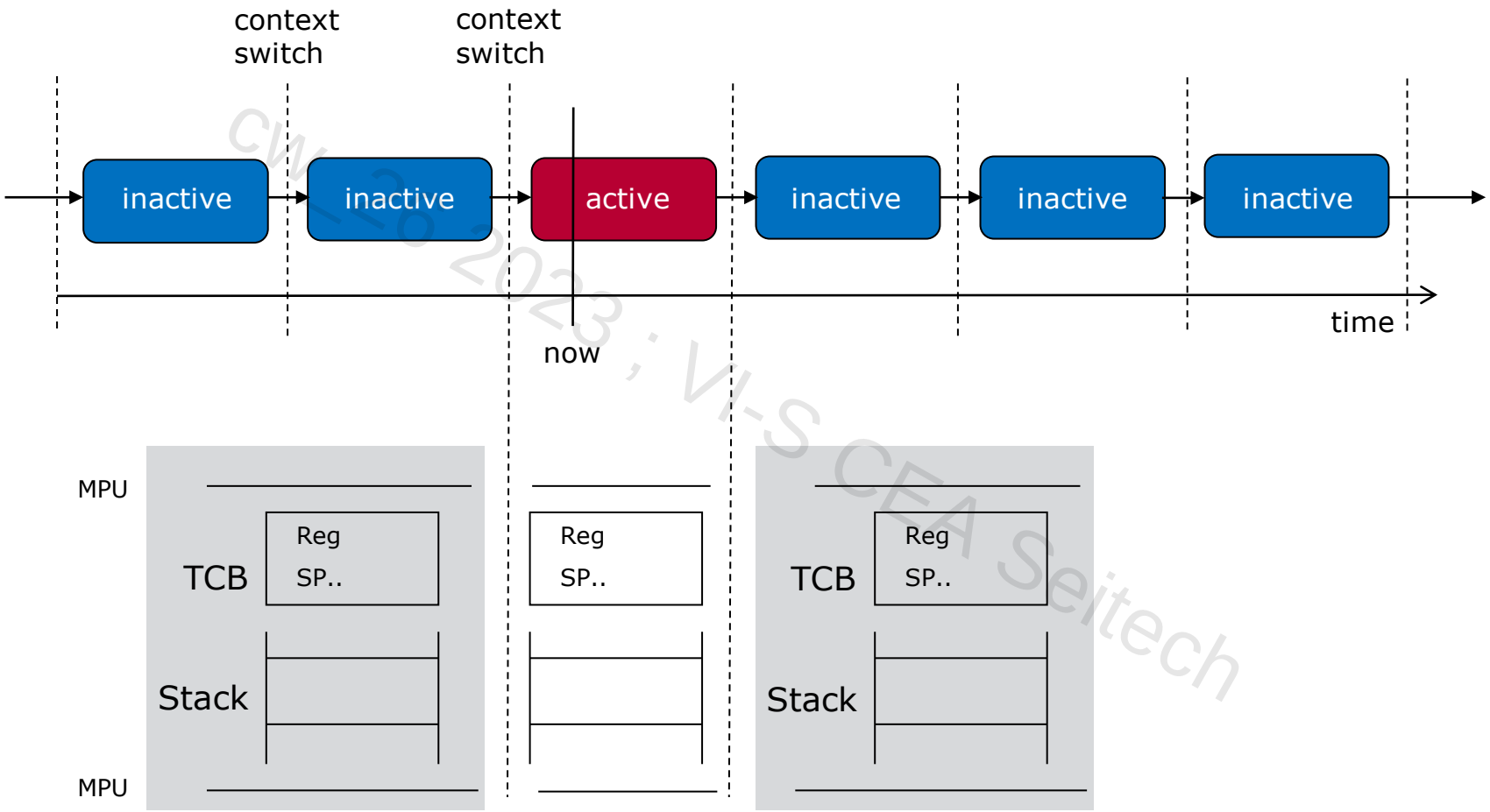
VECTOR

What is a context

- A “context” of a thread of execution (task/ISR) consists of
  - A set of processor core registers and their contents
    - Processor Status Word
    - Program Counter
    - Interrupt Enable Flags
    - Stack Pointer
    - …
  - MPU settings
  - CPU mode
  - preemptable
  - Stack memory and its current content
- At a time, one processor core has at most one active context
  - Active context can only be changed by the allocated program
- Inactive context is invariant

cw_26 2023 ; VI-S CEA Seitech

Context



cw 28 2023 ; VI-S CEA Seitech

Context switch

- A context switch occurs in several situations
  - Task switch
  - ISR entry
  - Call of OS services
  - Call of trusted functions
  - Hook routines

CW_26_2023 ; VI-S CEA Seitech

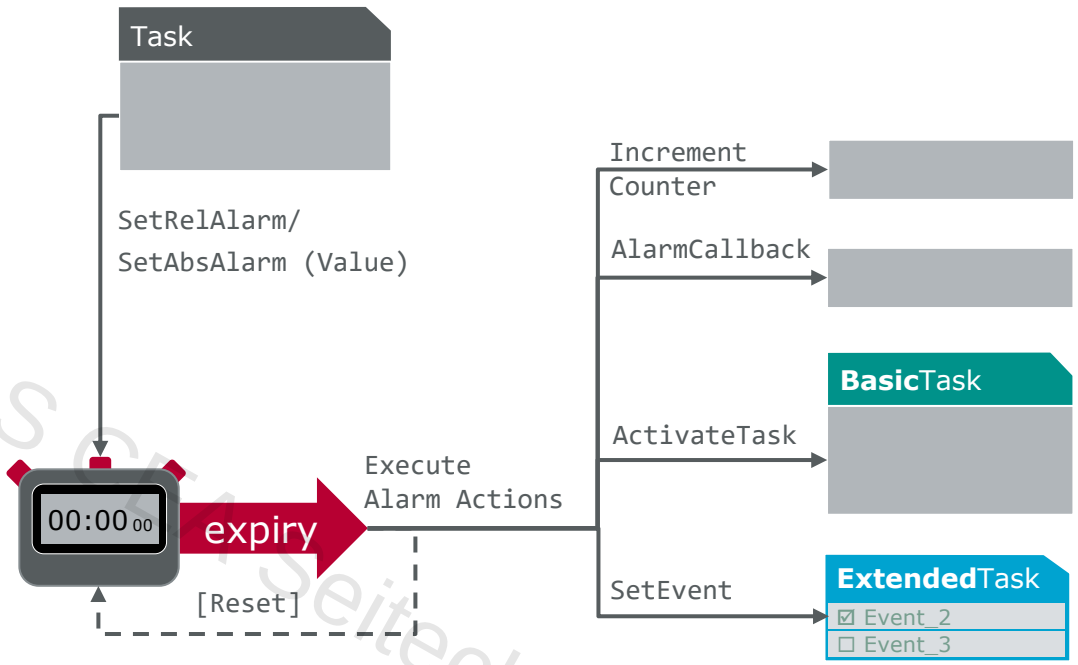
Agenda

- Overview
- Tasks
- **Alarms**
- OS counter
- Interrupts
- IOCs
- OS Resources
- Illustration
- Scalability of the OS
- Tool example
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

Alarms

- Alarms can be set to absolute or relative points of the system timer
  - The time to which the alarms are set is an argument to the Alarm API
- You can
  - set alarms
    - Dynamically over API
    - Statically over configuration (AUTOSTART)
  - cancel alarms
- Once an alarm expires
  - An alarm action defines what happens
    - Activate a task
    - Set an event
    - Call a callback function
    - Increment an OS counter



If you set a series of alarms, the system timer may increase in the midst of setting the alarms of the series. This results in different starting times of the alarms. You can prevent this if

- a so called "High Resolution Timer" is used
- schedule tables are used

ai_26 2023 ; VI-S ert Seitech

Agenda

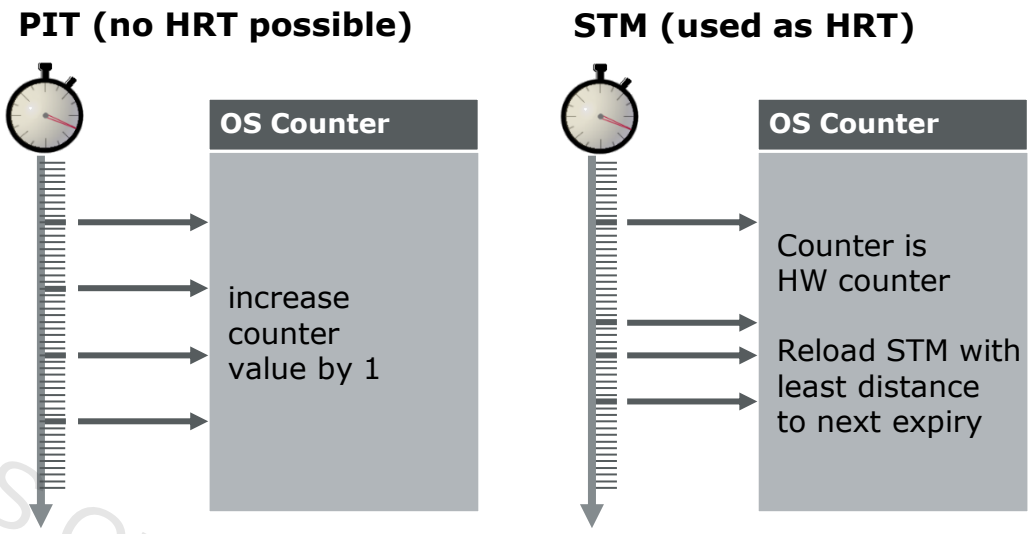
- Overview
- Tasks
- Alarms
- **OS counter**
- Interrupts
- IOCs
- OS Resources
- Illustration
- Scalability of the OS
- Tool example
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

OS counter / PIT or High resolution timer

OS counters are used to provide a time base "System Timer" for the operating system

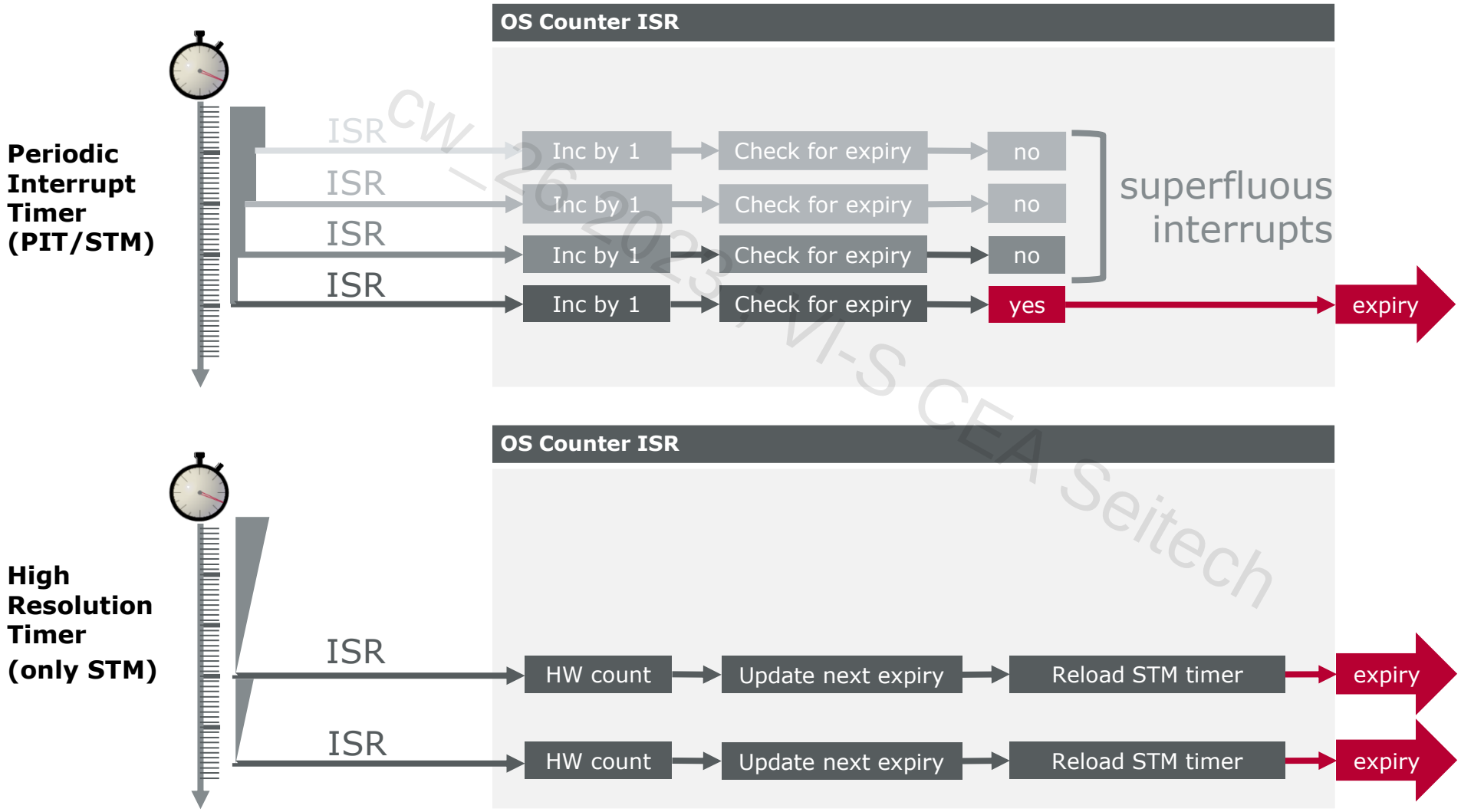
- Counter Driver (HARDWARE)
  - PIT Periodical Interrupt Timer
    - No HRT possible: OsDriverHighResolution = FALSE
    - Equidistant periodic ticks
    - Increment counter by a tick
    - High precision results in a high interrupt frequency
  - STM System Timer Module
    - HRT supported: OsDriverHighResolution is
      - TRUE: HRT supported
      - FALSE: STM is working periodic like a PIT
    - High precision is possible
    - Aperiodic ticks
    - Increment counter by last interval duration
    - Program timer to next expiry
    - High precision possible with low interrupt frequency



	PIT	HRT
IRQ	periodical	On demand
Precision (Alarms, schedule tables)	Multiples of OsSecondsPerTick	Any time the timer hardware can provide
Interrupt Load	Constant equally distributed	No equally distributed, bursts possible

CW 26 2023 ; VI-SCEA Seitech

Periodical behavior vs High Resolution Timer



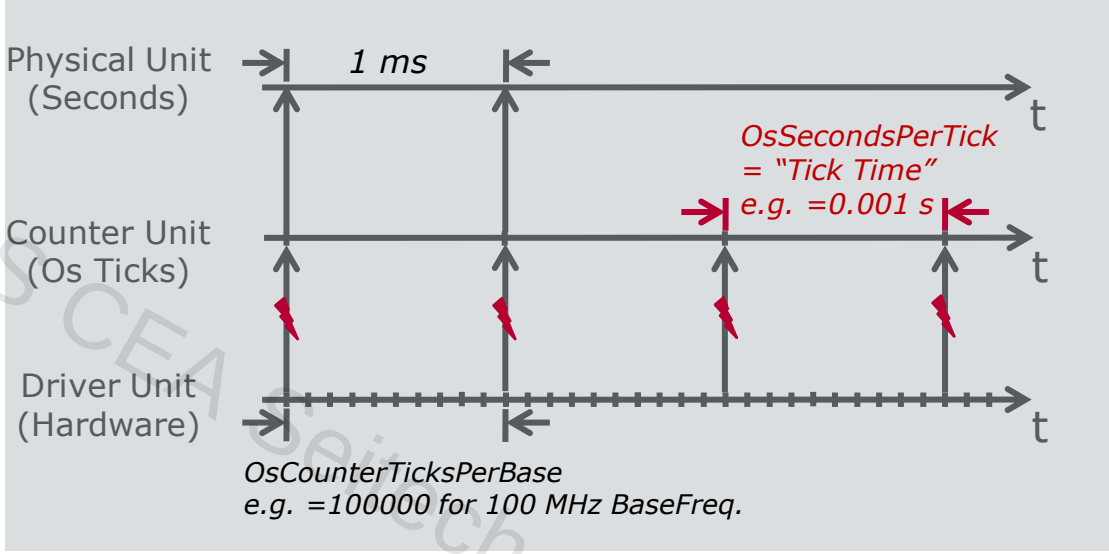
CW 26 2023, VI-S CEA Seitech

OS time base parameters

- Time base
  - *OsSecondsPerTick [s]* resolution for SW counter increments
- Counter increments each *OsSecondsPerTick*
- The OS provides macros to convert between OS ticks and seconds
- Counter Type can be
  - HARDWARE
    - Have to select a driver (PIT/STM…)
    - Hardware advances OS counter over interrupts
  - SOFTWARE
    - Can use the `IncrementCounter` API to advance the System Time
- PIT:
  - In every tick, the counter increments by 1
- HRT:
  - TicksPerBase =1 → OS Counter increments by HW Base

Illustration of the OC counter configuration

$$OsCounterTicksPerBase = OsSecondsPerTick[s] \cdot BaseFreq[Hz]$$



Timer Interrupt

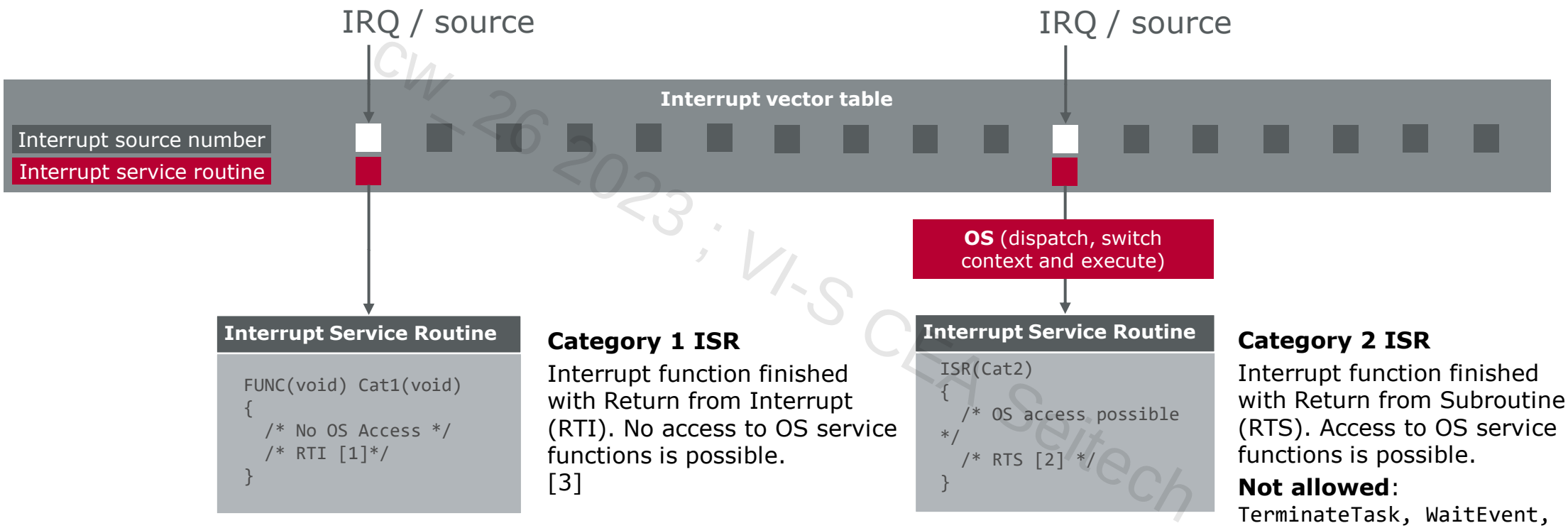
cw_26_2023 ; VI-S CEA Seitech

Agenda

- Overview
- Tasks
- Alarms
- OS counter
- ▶ **Interrupts**
- IOCs
- OS Resources
- Illustration
- Scalability of the OS
- Tool example
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

Interrupt categories



Category 1 ISR
Interrupt function finished with Return from Interrupt (RTI). No access to OS service functions is possible.
[3]

Category 2 ISR
Interrupt function finished with Return from Subroutine (RTS). Access to OS service functions is possible.
Not allowed:
`TerminateTask, WaitEvent, ClearEvent, Schedule, ChainTask.`

[3] OS locks all interrupts during context switch. This may take a few µs. For whom this is still too much, we also have "CAT0" Interrupts. (Vector specific implementation) Here, the OS interrupt locking times are significantly shorter.

Interrupt sources must be enabled/disabled over OS API Os_Enable/DisableInterruptSource

An **Interrupt** can use OS functionality or not. This depends on the interrupt category.

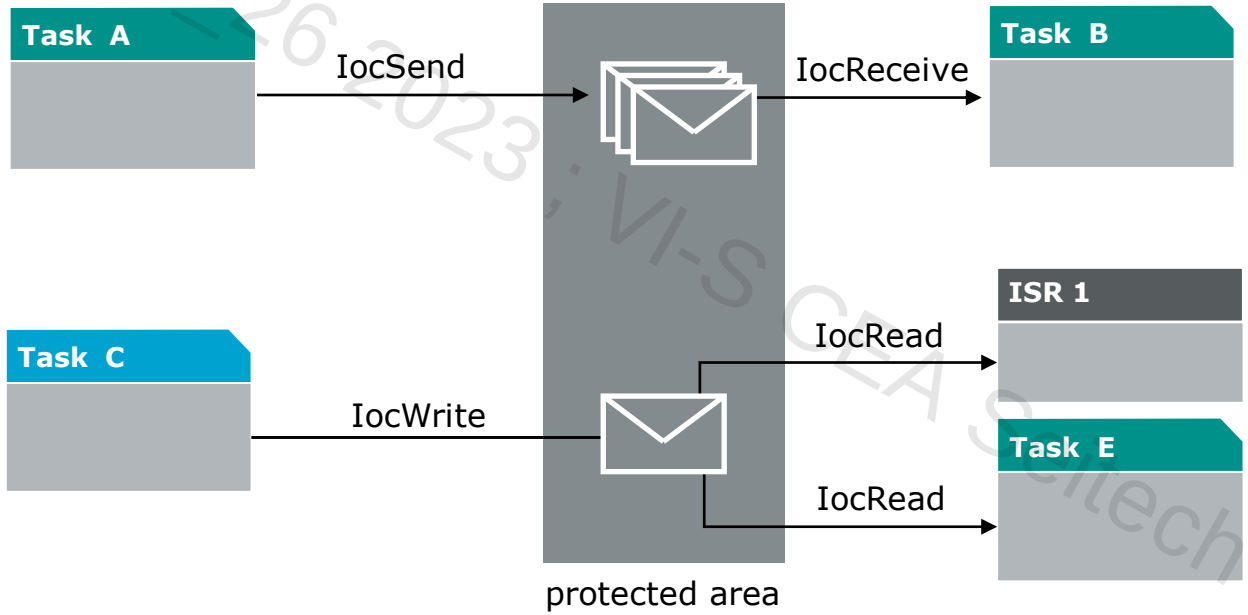
cw 26 2023 ; VI-S CEA Seitech

Impact of the Interrupt category on the ECU software

properties	CAT1 ISR	CAT2 ISR
Context	Runs in any context currently active	Runs in its own dedicated context
Stack usage	You'll need to reserve a safety margin of RAM for all CAT1 ISRs on all stacks of all tasks per core. Inefficient RAM usage.	You'll be able to decide on the amount of stack per ISR. Efficient RAM usage possible.
Memory protection	Stack overflow can lead to a protection trap inside all OS applications on the core. Memory corruption may remain undetected	Both stack overflow and memory corruption can lead to protection trap inside its OS application. Memory corruption attempts are detectable
Processor mode	Runs regardless of supervisor/user mode. Unintended access to some processor registers / peripherals possible.	Runs in the processor mode defined by the containing OS application. No way to affect the processor registers /peripherals in an unintended way.
impact		
Safety projects (mixed ASIL): requirements for ISRs	In safety scenarios with memory protection, all CAT1 ISRs must be implemented according to the highest ASIL in the system. Non ASIL ISRs must be qualified. If this is not done, memory corruption can happen and remain undetected.	CAT2 ISRs can be used for all ASILs/QM. Implementation according to the respective ASIL. Memory corruption can be detected.
How to decouple from ISR into TASK context	Can not be used for decoupling from ISR to TASK context using OS mechanisms. No task can be triggered. Only flags work. (DEFERRED operation)	Can be used for IMMEDIATE decoupling from ISR to TASK using OS mechanisms. Tasks can be triggered. That is the way the RTE works.

CVUES 2022 M-S CEP Seitner

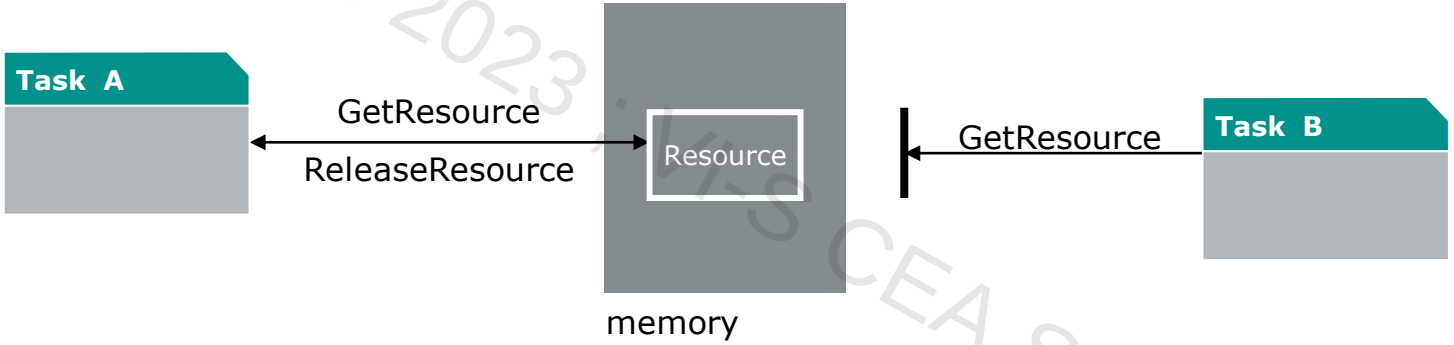
Inter OS Application Communicator (IOC)



cw 26 2023 ; VI-S CEA Sotech

An **IOC** can have several writers and several readers. Therefore IOCs support n:m data transfer across several OS applications.

Deadlock prevention using OS Resources



cw_26 2023 ; IVFS CEA Seitech

A OS **Resource** can be used to protect against concurrent access by at least two concurrent tasks without using Interrupt locks. Priority Ceiling Protocol.

Deadlock prevention using OS Resources

Resource management coordinates the access to shared resources:

- memory (RAM)
- hardware components
- application program sequences (critical sections)
- elements of the operating system (Scheduler)

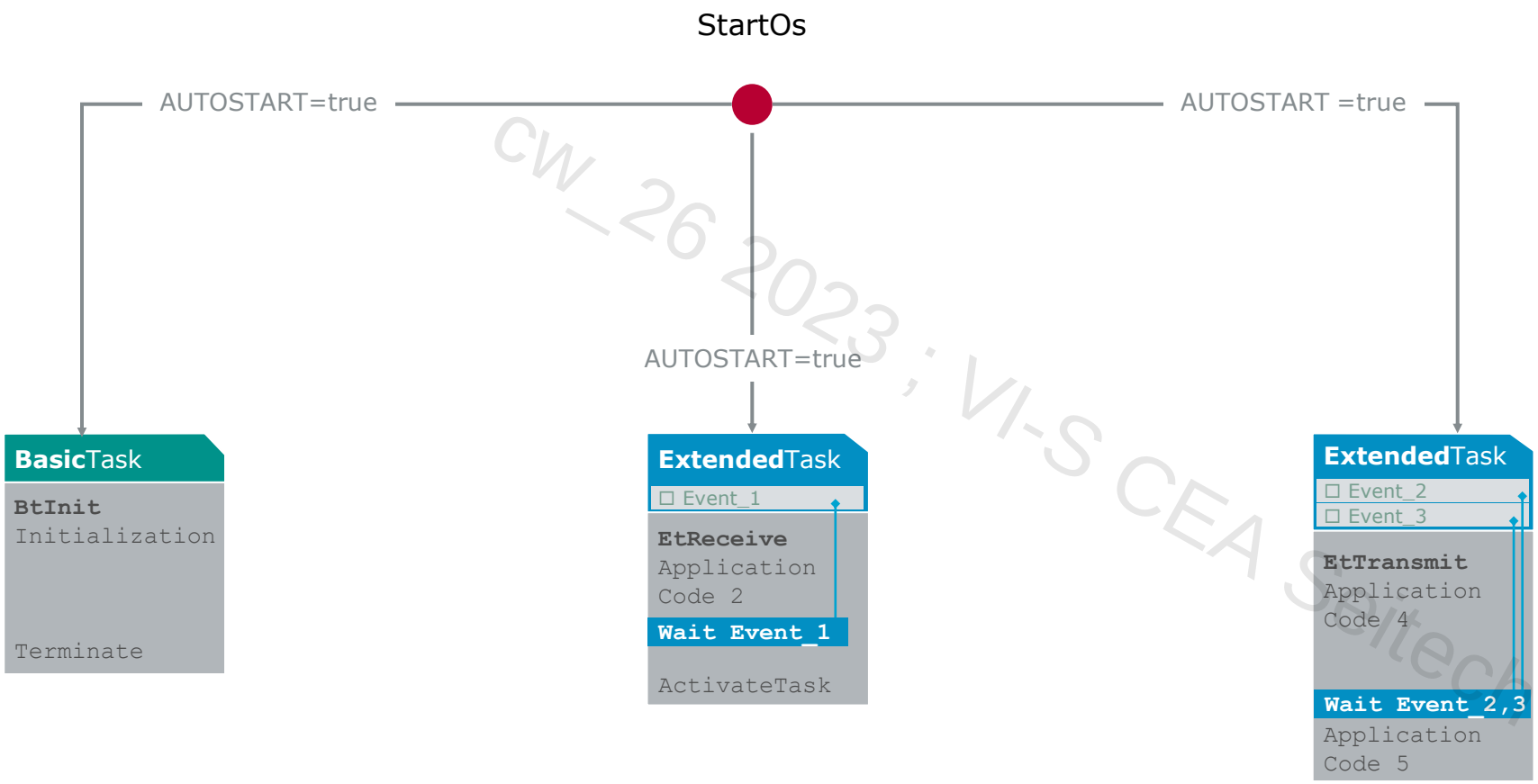
OSEK/VDX and AUTOSAR OS guarantee that

- a resource is never occupied by two users simultaneously
- priority inversion is excluded
- deadlocks are prevented
- access to a resource never ends in a wait state

Used for Exclusive Areas in AUTOSAR!

cw_26 2022 ; VI-S CEA Seitech

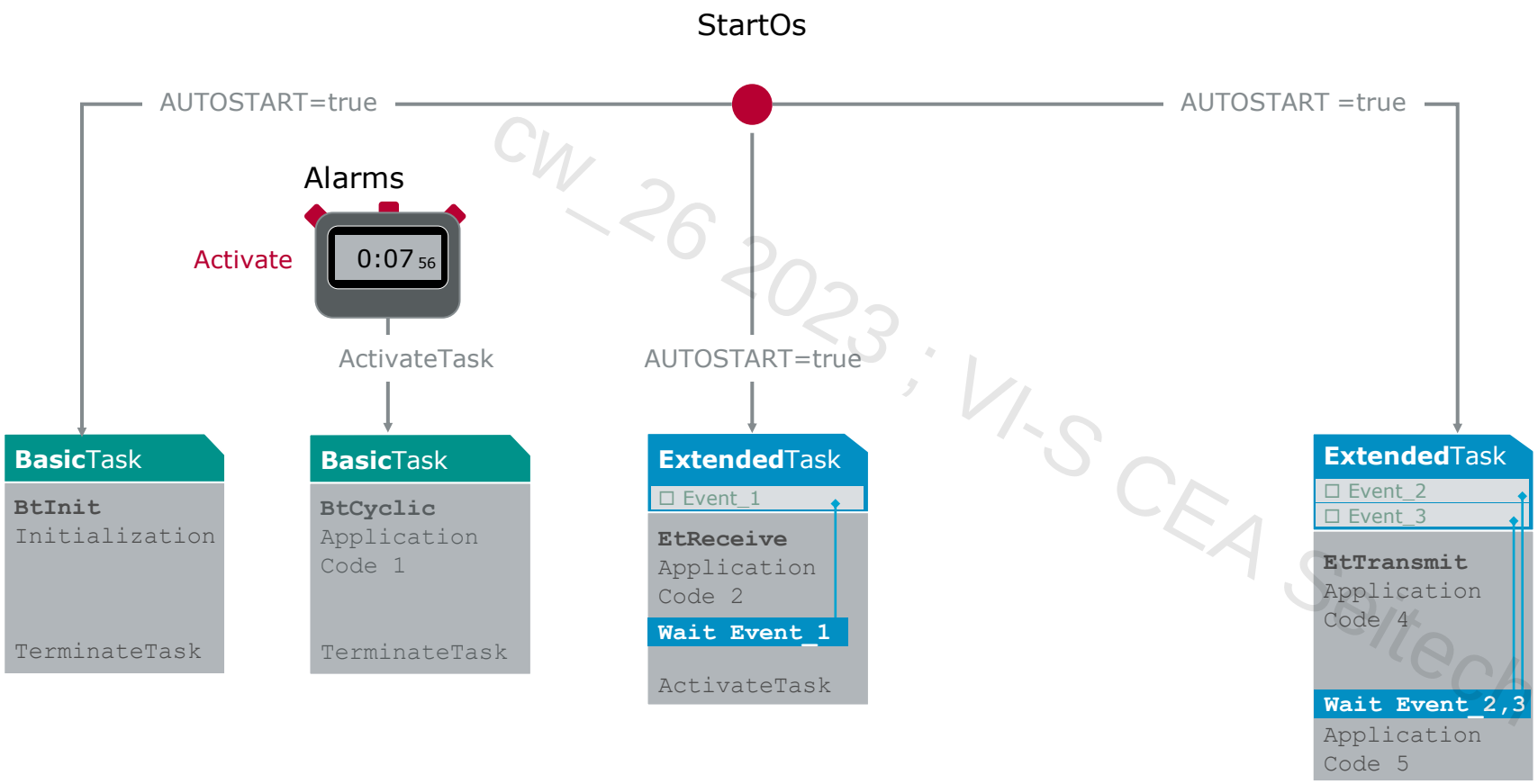
Interaction of OS Elements



cw_26 2023 ; VI-S CEA Saitech

Activation of Tasks after System Startup

Interaction of OS Elements



cw_26 2023 ; VI-S CEA Saitech

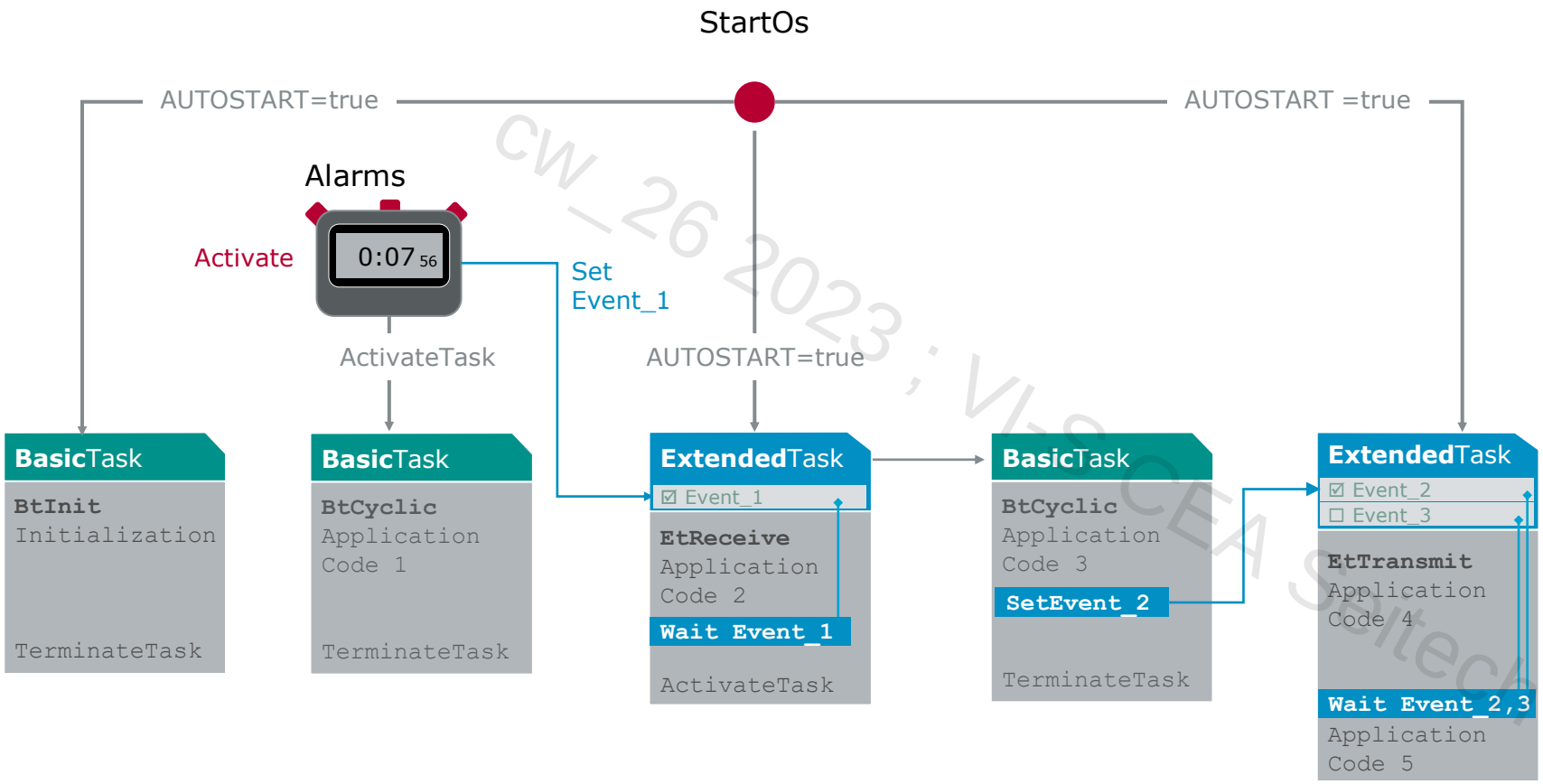
Alarm Triggered activation of Task

Alarm Activation

An Alarm can be activated via:

- System start
- User code

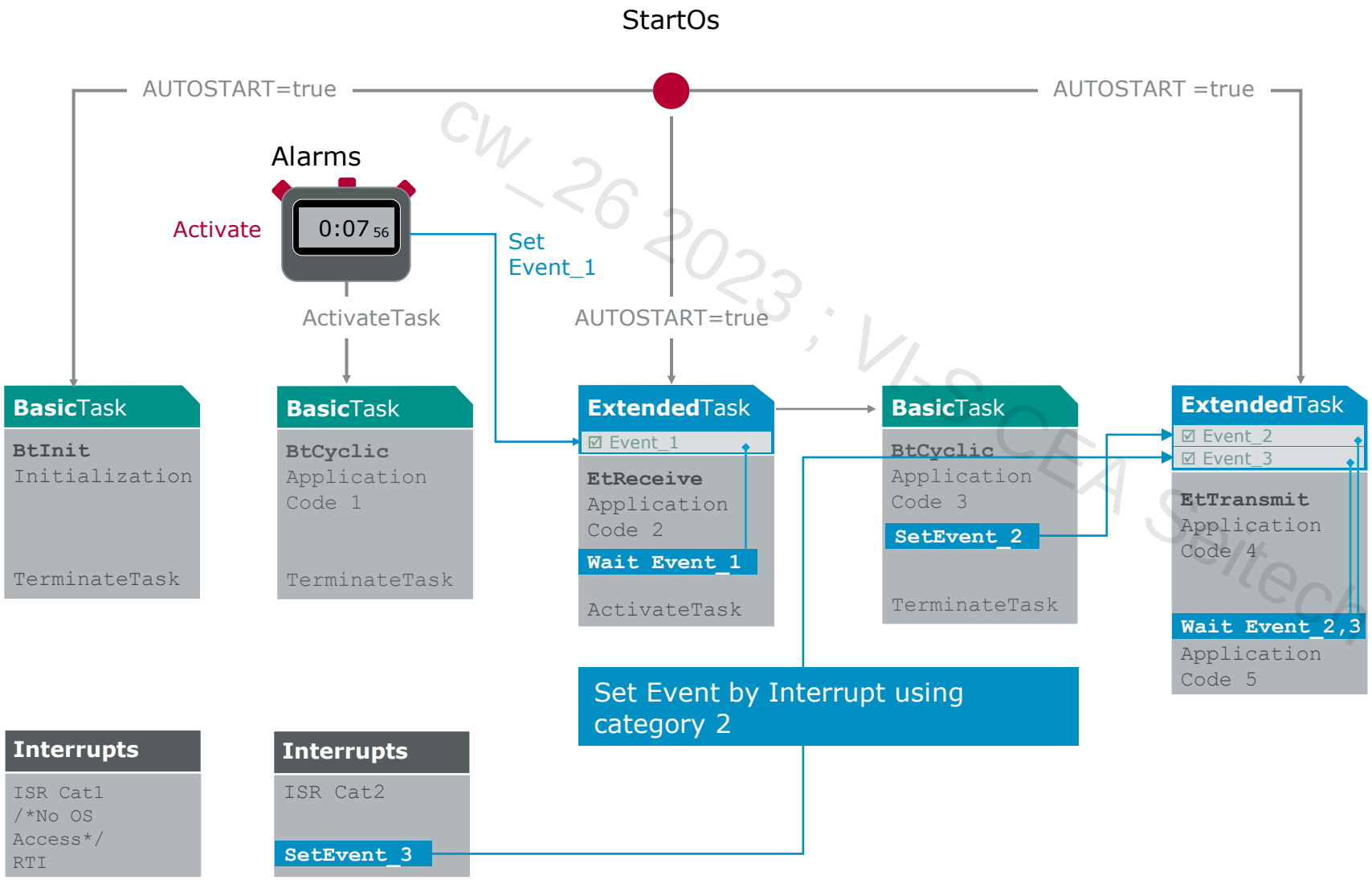
Interaction of OS Elements



Events set by Alarm and by a Task

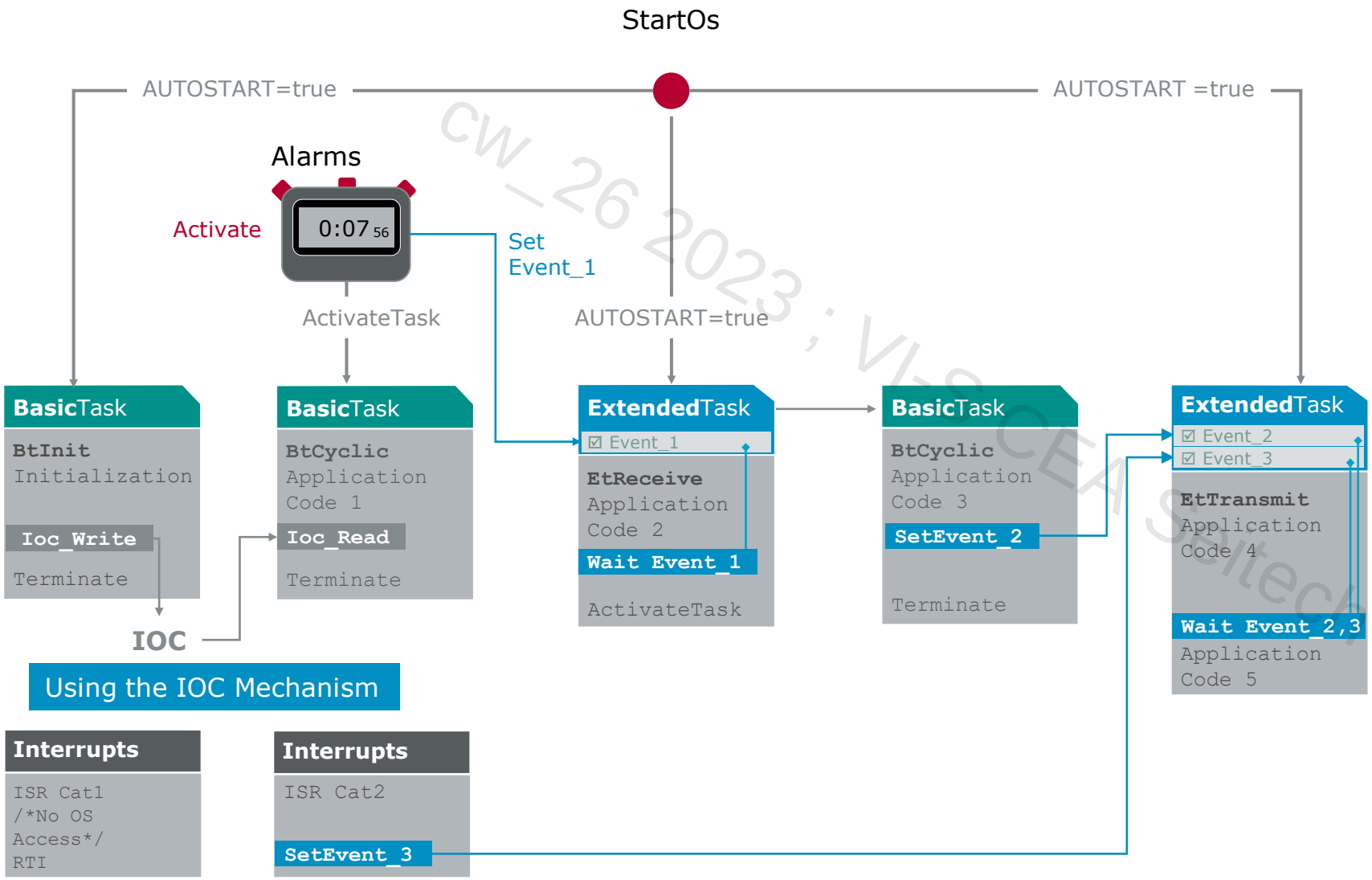
cw_26 2023 ; VI-CEA Saitech

Interaction of OS Elements



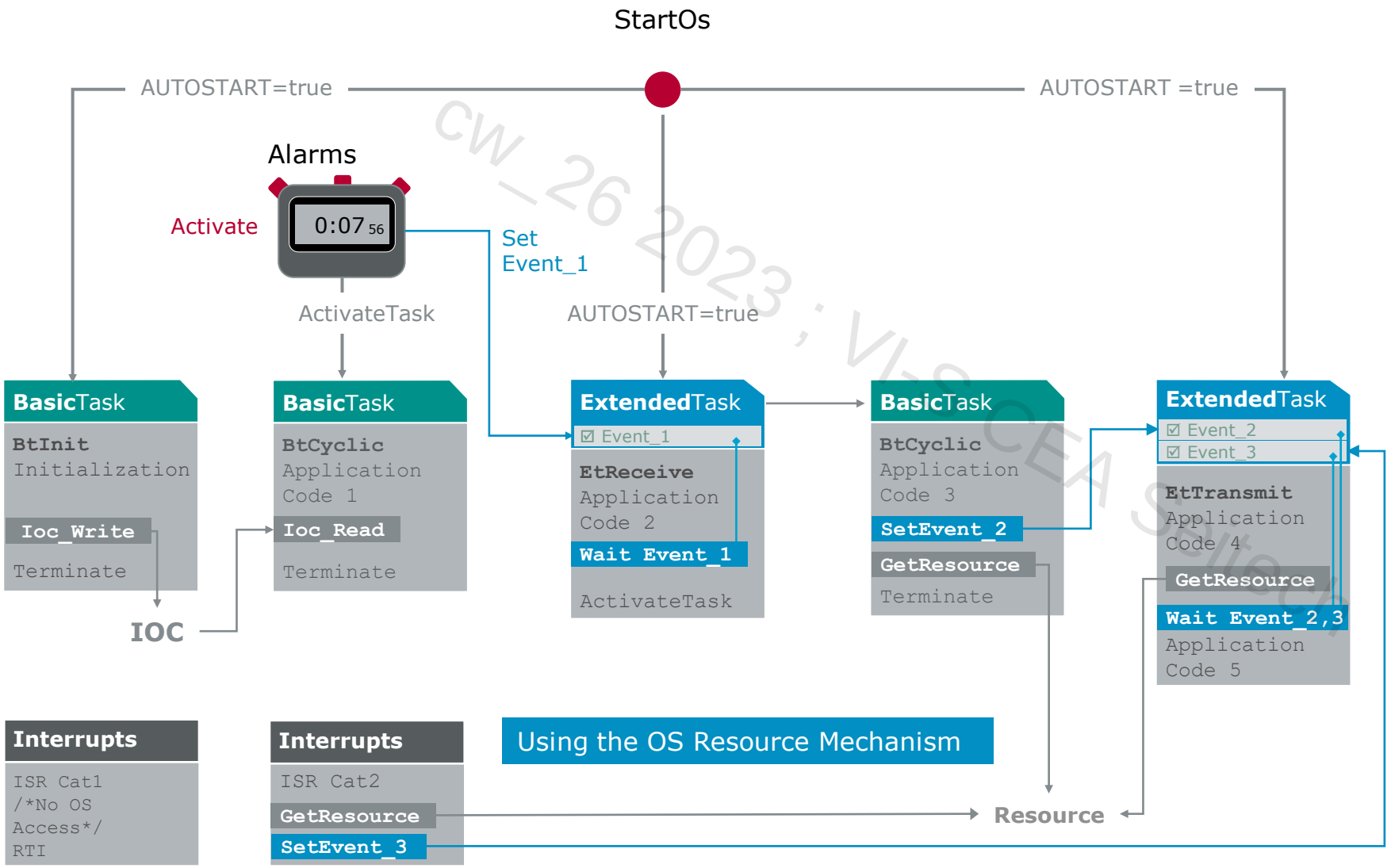
cw_26 2023 ; VI-SCEA Sotech

Interaction of OS Elements



cw_26 2023 ; VI-SCEF Sotech

Interaction of OS Elements



cw_26 2023 ; VI-CEA Sellect

Agenda

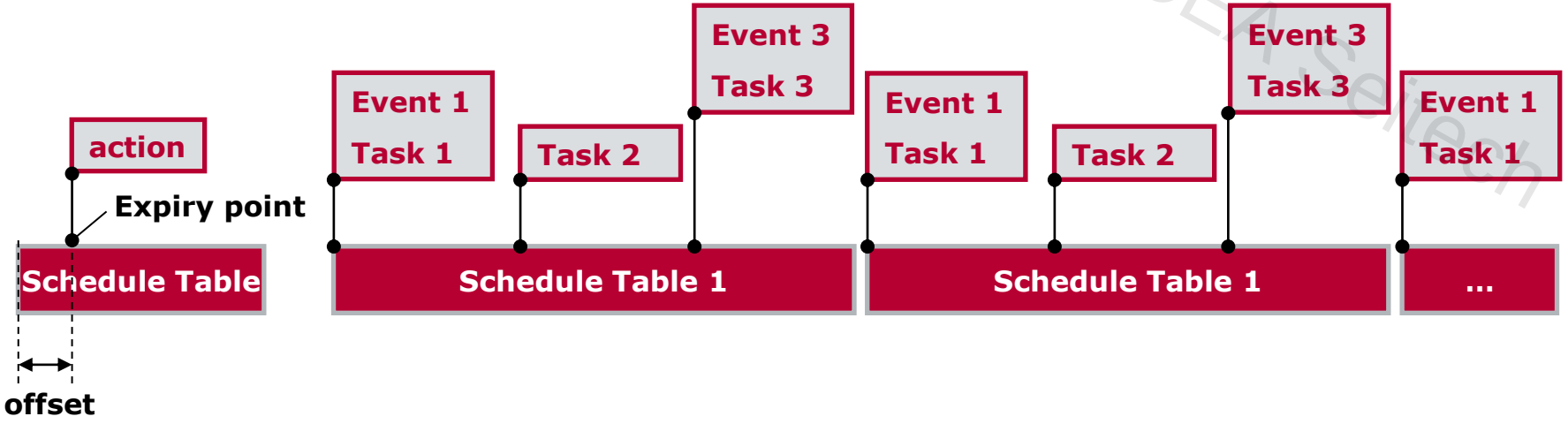
- Overview
- Tasks
- Alarms
- OS counter
- Interrupts
- IOCs
- OS Resources
- Illustration
- ▶ **Scalability of the OS**
- Tool example
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

SC1: Schedule Tables

- Definition of sequences of actions with a fixed time distance
- Can be executed **once** or **repeatedly**
- Can be autostarted or activated by the application
- Local task execution can be synchronized to a global time base
  - E.g., to the time base of a FlexRay bus (cycle start)
- Multiple schedule tables can be active at the same time

CW 20 2023, VI-S CEA Seitech



Each schedule has so called **Expiry Points** which occur within the **schedule table duration**.

Expiry Points are used to cause "**actions**" as soon as they occur.

An action is either a **task activation** or a **SetEvent()** operation or both.

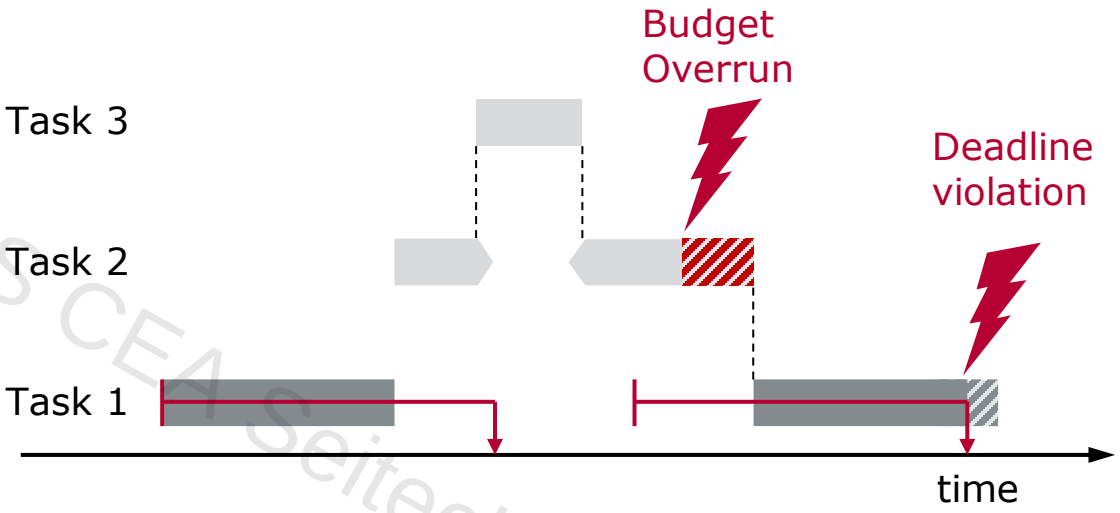
Expiry Points are positioned relative to the start of the schedule.

Each Expiry Point is signified within the schedule table by an individual "**offset**".

SC2: Timing Protection

- Execution budgets are assigned to tasks and monitored
- If the budget is exceeded the Protection hook is called
- Similarly, inter-arrival-times and resource locking times are monitored

- Timing Protection vs. Watchdog
  - Timing protection does not detect deadline violations!
  - Detects causes of deadline violations earlier than watchdog
  - Timing protection is limited to tasks / ISR 2 (ISRs of type 1 bypass the timing protection)



cw_26_2023 ; VI-5 CEA Seitech

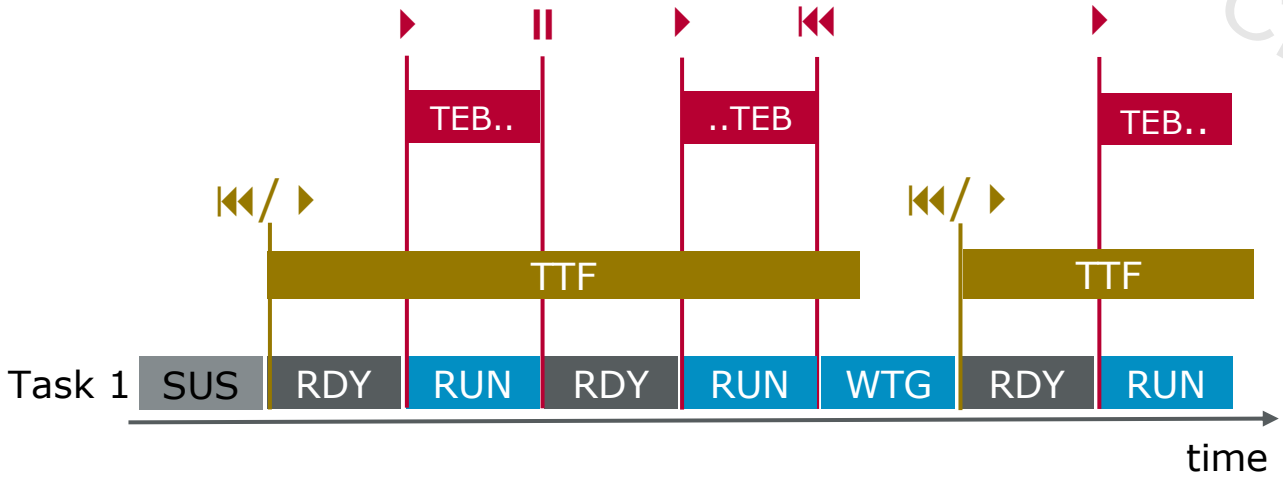
SC2: Timing protection (execution time / inter arrival time)

OsTaskExecutionBudget (TEB)

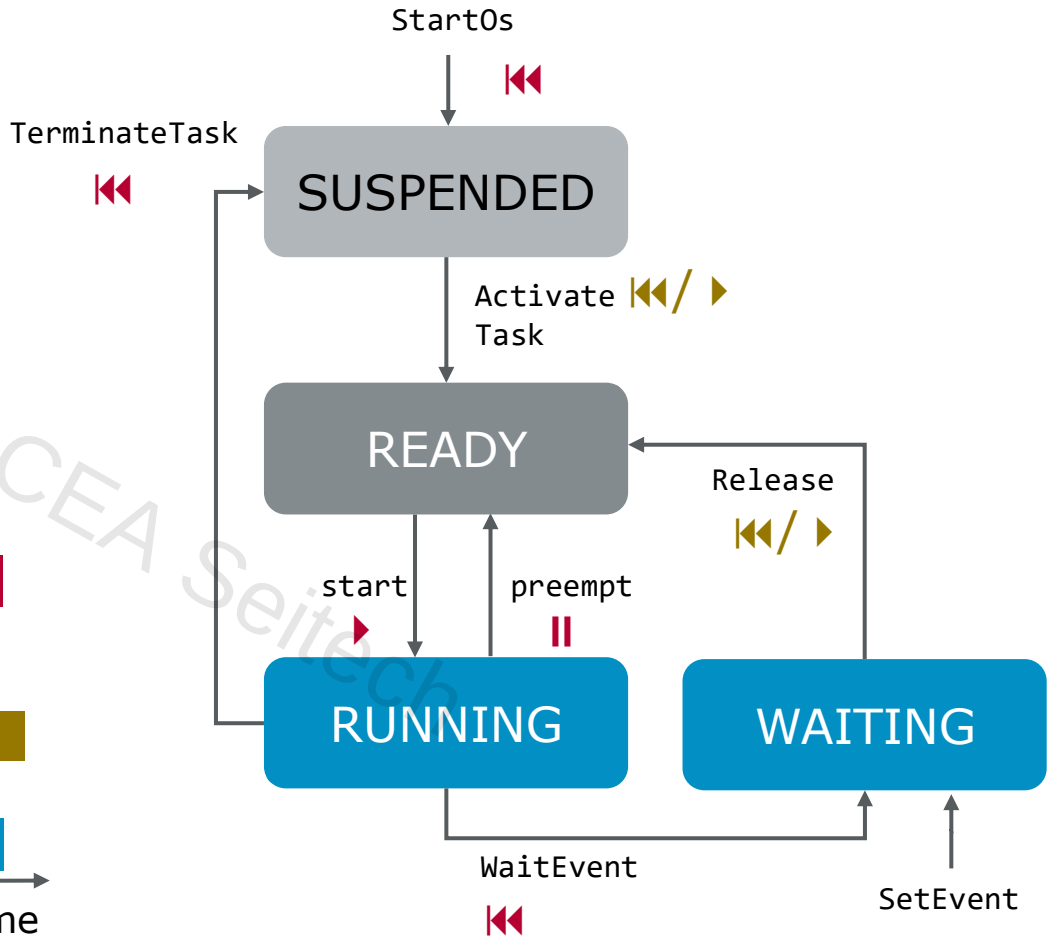
- maximum allowed execution time of the task
- To detect too long execution times

OsTaskTimeFrame (TTF)

- The minimum inter-arrival time between activations and/or releases of a task
- To detect too frequent activations



TEB Task Execution Budget TTF Task Time Frame



OW_26_2023 ; VI-S CEA Seitech

SC2: Timing protection (execution time / inter arrival time)

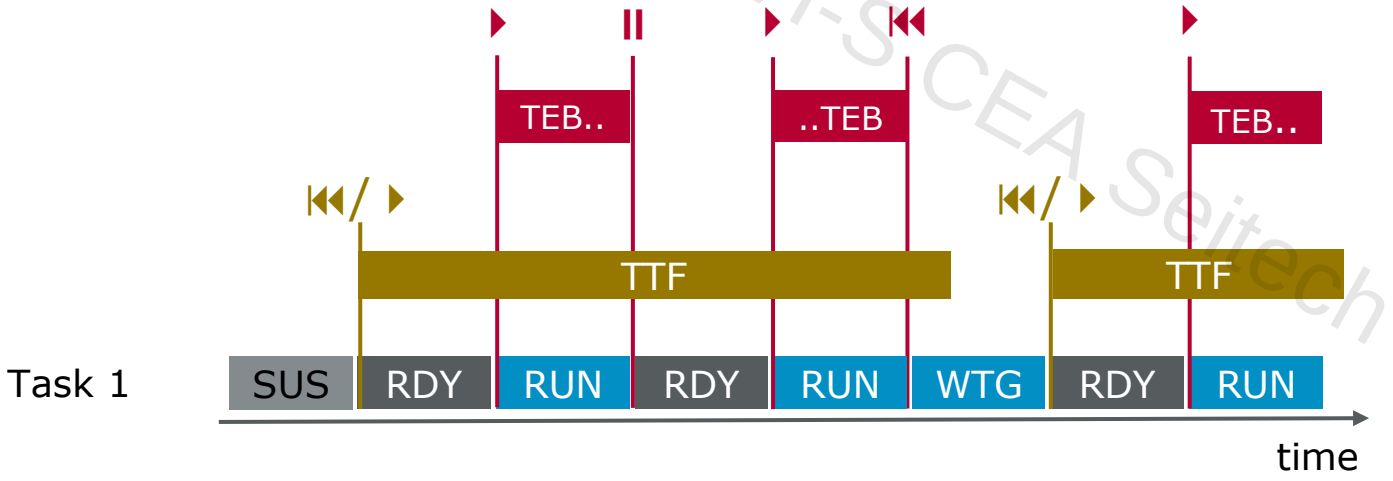
OsTaskExecutionBudget (TEB)

- maximum allowed execution time of the task
- To detect too long execution times

OsTaskTimeFrame (TTF)

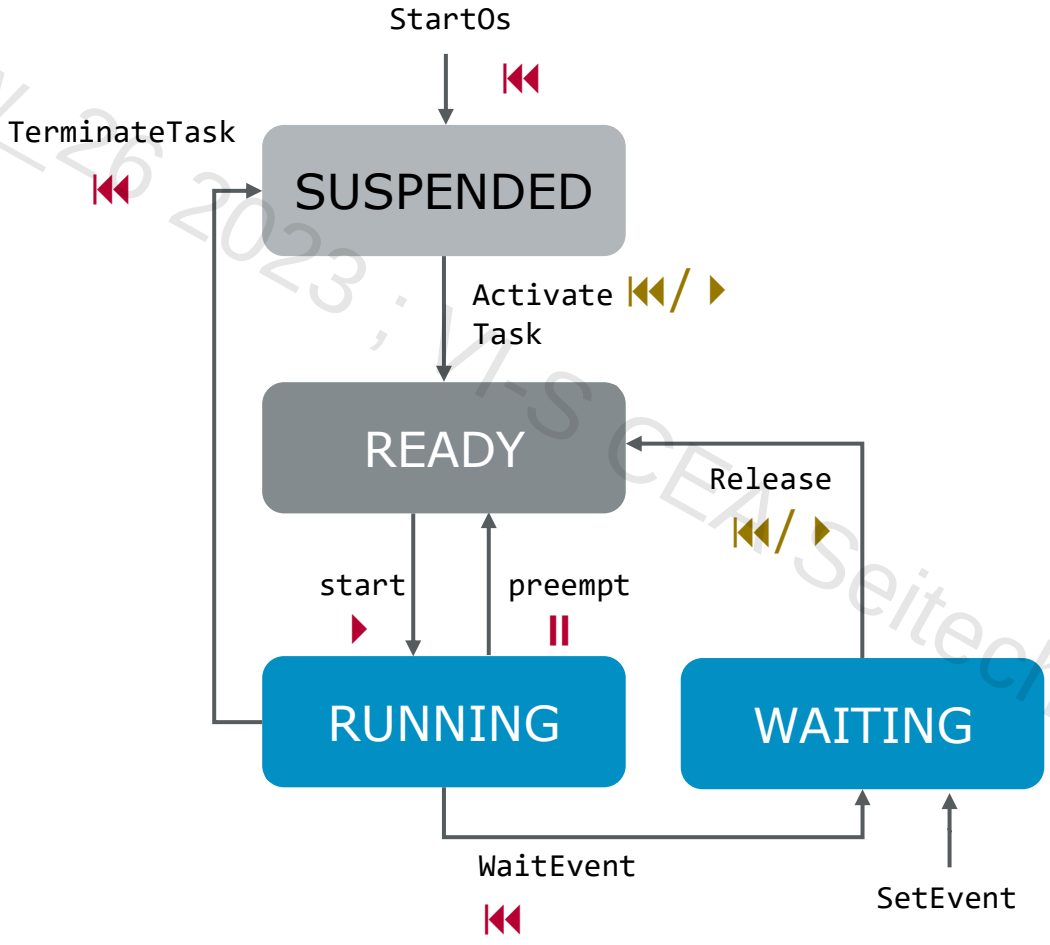
- The minimum inter-arrival time between activations and/or releases of a task
- To detect too frequent activations

OW_26 2023 ; VI-SCEA Seitech



TEB Task Execution Budget
TTF Task Time Frame

SC2: Task Execution Budget / Time Frame



cw_26_2023 ; ITS CEN Seitech

TEB Task Execution Budget
TTF Task Time Frame

SC3: Memory Protection

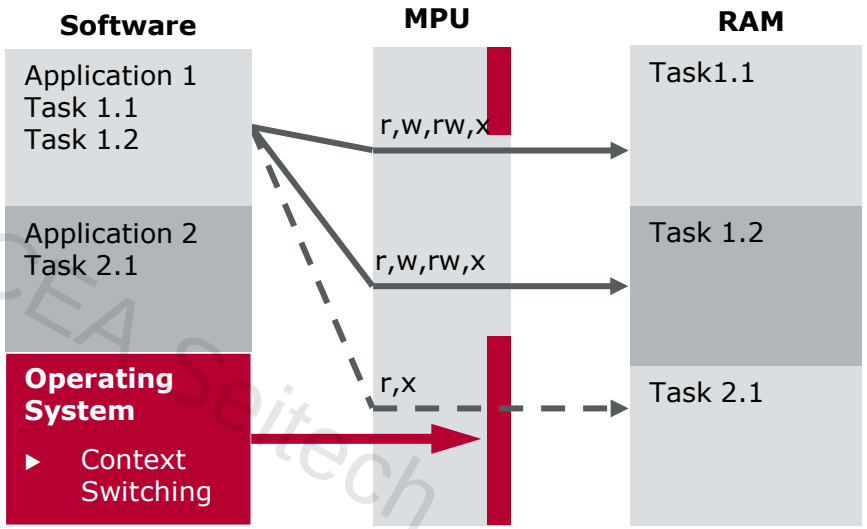
Motivation and background

- Easy integration of applications
  - Faulty code of one application has no influence on others
- Hardware support needed (MPU)
- Stack protection
  - Immediate detection of stack overflow
  - Constituent parts of an OS-Application
- Data protection
  - Private data sections of OS-Applications are shared by all its tasks and ISRs
- Code protection
  - Protect commonly used code (e.g. shared libraries) to prevent memory, timing or service violation.

ch_26 2023 ; VI-S CEA Seltech

SC3: OS-Applications

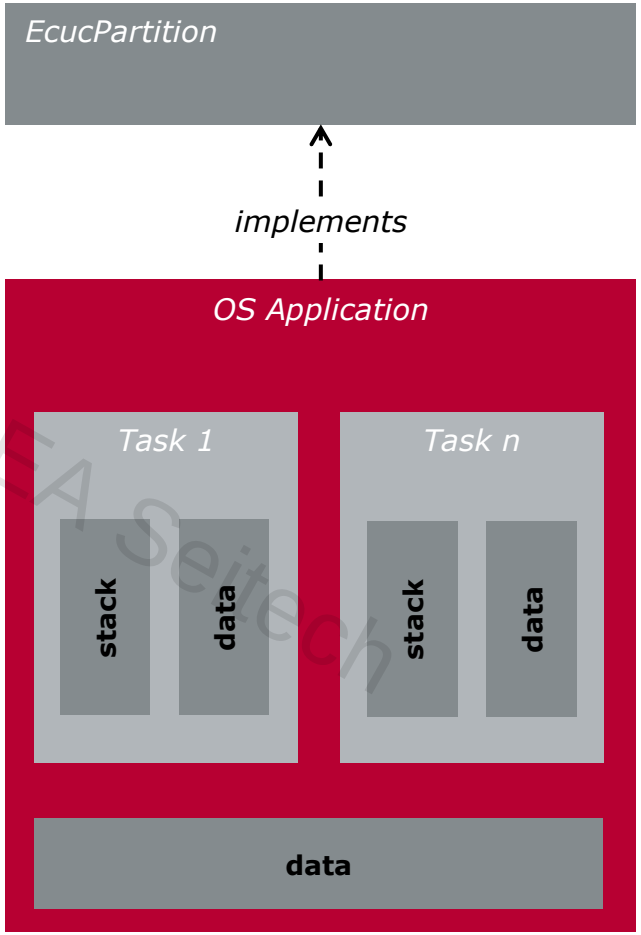
- Collection of cohesive functional units
  - Tasks, ISRs, alarms, hooks, etc.
- AUTOSAR OS shares systems resources between OS-Applications
- All objects belonging to the same OS-Application can access each other
- Access between OS-Applications may be granted in configuration
- Classes of OS-Applications
  - **Trusted**
    - Allowed to run **without** protection
    - Unrestricted access to memory / APIs
  - **Non-Trusted**
    - Run only with protection
    - Only limited access to memory / APIs



CW_26_2023, VI-S CEA Seitech

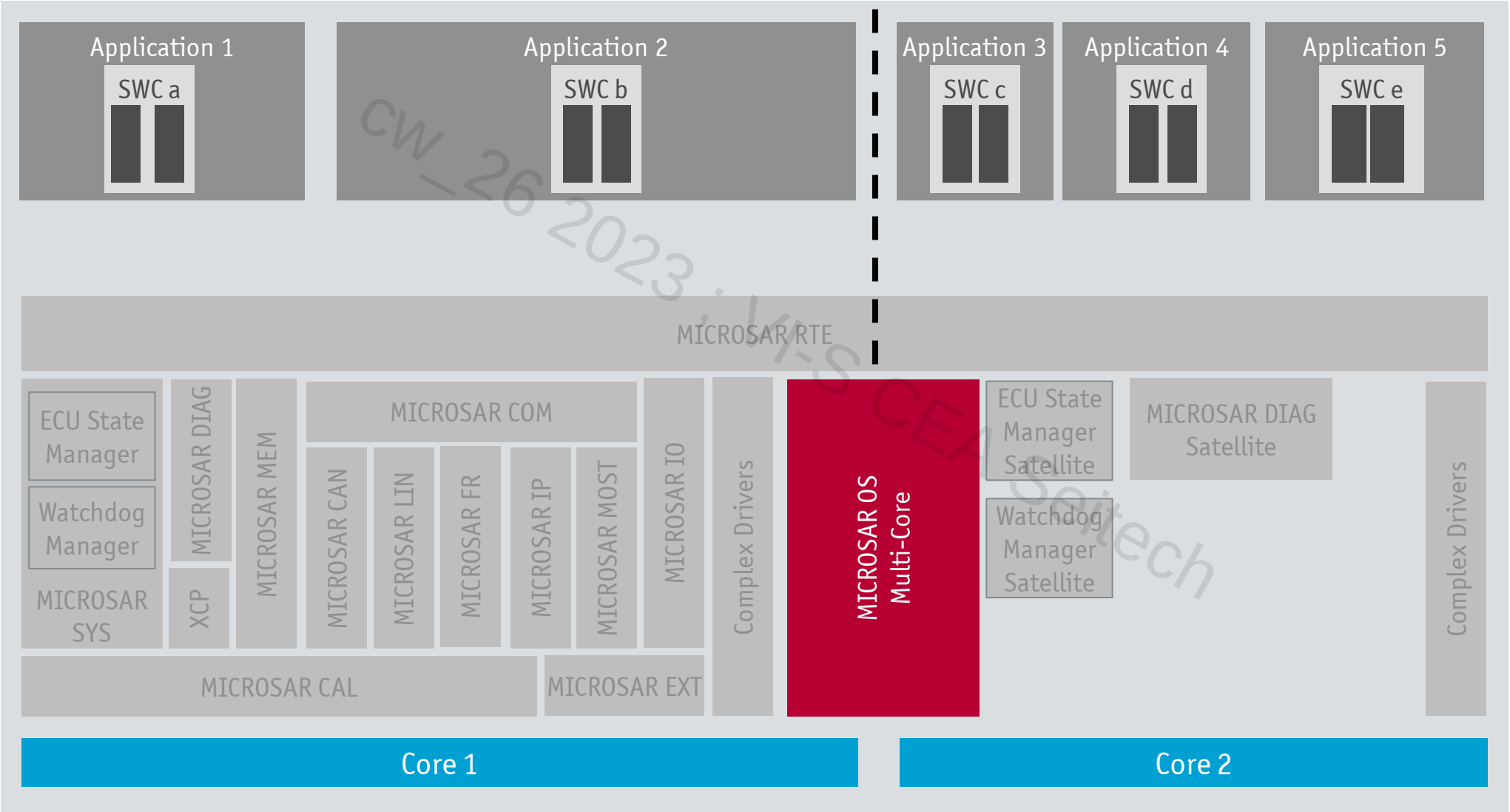
OS Application

- References and implements an *EcucPartition*
- Manages access to the µC resources
  - Runtime (CPU core)
  - Memory access
- Collection of OS objects which shall use these resources
  - Tasks
  - Interrupts
  - Alarms
  - Events
  - Hooks…
- An OS Application
  - summarizes all elements having access to each other
  - has a data region that can be accessed by all its tasks



cw_26_2023 ; VI-S CEA Seitech

Multi-core operating system



cw_26_2023 ; V-SCEP seitech

Multi-core operating system

IOC: Inter-OS-Application Communicator

For CDDs (Complex Device Drivers) and Multi-Core:

- BSW resides on the Master core (Core0). If the CDD needs to access standardized interfaces of the BSW, it needs to reside on the same core.
- In case a CDD resides on a Slave core (Core1), it can use the normal port mechanisms to access AUTOSAR interfaces and standardized AUTOSAR interfaces. This involves the RTE, which uses the IOC mechanism of the operating system to transfer requests to the master core.
- Proxy solution: However, if the CDD needs to access standardized interfaces of the BSW and does not reside on the master core, a stub part of the CDD needs to be implemented on the master core, and communication needs to be managed locally in the CDD using the IOC mechanism of the operating system. Additionally, in this case the initialization part of the CDD also needs to reside in the stub part on the master core.

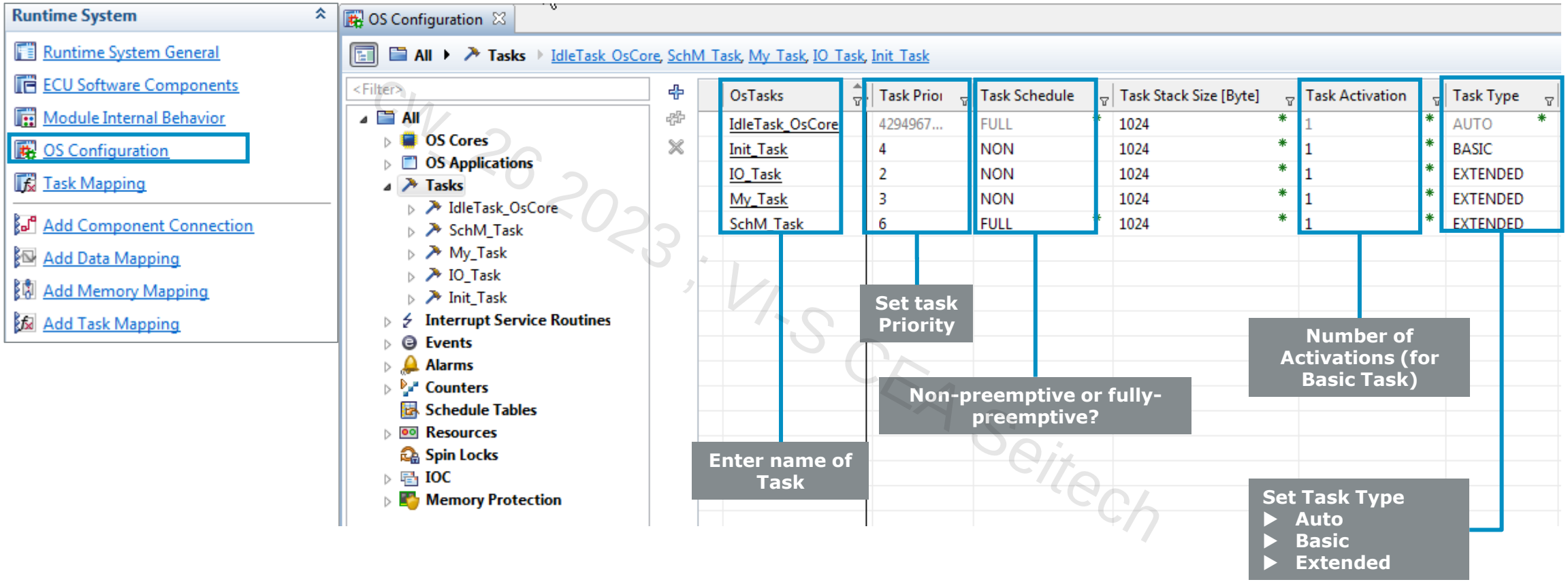
CW 28 2023, Vu Ngoc Seitech

Agenda

- Overview
- Tasks
- Alarms
- OS counter
- Interrupts
- IOCs
- OS Resources
- Illustration
- Scalability of the OS
- ▶ **Tool example**
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

OS in Configurator Pro



CW 26 2023 ; VIRSCEF Seitech

The RTE generator decides automatically how a task must be treated (by the Function Trigger to Task mapping)

- Application task, BSW Scheduler task, non-RTE task

Runnables in DaVinci Configurator Pro

ECU Software Components

All ▸ Application Components ▸ CpApMySwc ▸ Task Mapping

<Filter>

5 of 35 function triggers are mapped. Use the Task Mapping Assistant to map the runnable entities.

- All
  - ECU Composition
    - Port Prototypes
    - Application Connectors
    - Data Mapping
  - Application Components
    - IoHwAb_P
    - CpSaLightDimControl
    - CpSaInteriorLightFront
    - CpSaDoorFrontRight
    - CpSaDoorFrontLeft
    - CpApMySwc
      - Port Prototypes
      - Application Connectors
      - Service Connectors
      - Task Mapping
      - Exclusive Area Implementation
      - Data Mapping

Triggered Function	Function Trigger
CSDataServices_Data_Example_DIDData_ed123411_ReadData	On Operation Invocation CSDataServices_Data_Example_DIDData_ed123411.ReadData
CSDataServices_Data_New_Data_Object_14_2b55f702_ReadData	On Operation Invocation CSDataServices_Data_New_Data_Object_14_2b55f702.ReadData
CSDataServices_InitEvt_DTC_0x000002_InitMonitorForEvent	On Operation Invocation CSDataServices_InitEvt_DTC_0x000002.InitMonitorForEvent
CtApMySwc_Init	On Init
DataServices_Data_Example_Dynamic_Defined_Non_Periodic_Data_Identifier_ConditionCheckRead	On Operation Invocation DataServices_Data_Example_Dynamic_Defined_Non_Periodic_Data_Identifier.ConditionCh
DataServices_Data_Example_Dynamic_Defined_Non_Periodic_Data_Identifier_ReadData	On Operation Invocation DataServices_Data_Example_Dynamic_Defined_Non_Periodic_Data_Identifier.ReadData
DataServices_Data_Example_Dynamic_Defined_Non_Periodic_Data_Identifier_ReadDataLength	On Operation Invocation DataServices_Data_Example_Dynamic_Defined_Non_Periodic_Data_Identifier.ReadDataLe
DataServices_Data_Example_IO_Control_ConditionCheckRead	On Operation Invocation DataServices_Data_Example_IO_Control.ConditionCheckRead
DataServices_Data_Example_IO_Control_FreezeCurrentState	On Operation Invocation DataServices_Data_Example_IO_Control.FreezeCurrentState
DataServices_Data_Example_IO_Control_ReadData	On Operation Invocation DataServices_Data_Example_IO_Control.ReadData
DataServices_Data_Example_IO_Control_ResetToDefault	On Operation Invocation DataServices_Data_Example_IO_Control.ResetToDefault
DataServices_Data_Example_IO_Control_ReturnControlToECU	On Operation Invocation DataServices_Data_Example_IO_Control.ReturnControlToECU
DataServices_Data_Example_IO_Control_ShortTermAdjustment	On Operation Invocation DataServices_Data_Example_IO_Control.ShortTermAdjustment
DataServices_Data_Example_PeriodicData_ConditionCheckRead	On Operation Invocation DataServices_Data_Example_PeriodicData.ConditionCheckRead
DataServices_Data_Example_PeriodicData_ReadData	On Operation Invocation DataServices_Data_Example_PeriodicData.ReadData

Runnables appear as "Triggered Functions"

Trigger conditions

CWY 2025 Vi-S CEA Seitech

Please note the Access Point cannot be changed or viewed in Configurator. The Trigger also cannot be changed in Configurator. These are design aspects and have to be changed in Developer.

Task mapping using the ECU Software Components view

Runtime System
- Runtime System General
- ECU Software Components
- Module Internal Behavior
- OS Configuration
- Task Mapping
- Add Component Connection
- Add Data Mapping
- Add Memory Mapping
- Add Task Mapping

ECU Software Components

All ▸ Application Components ▸ CpApMySwc ▸ Task Mapping

5 of 12 function triggers are mapped. Use the Task Mapping Assistant to map the runnable entities.

- All
  - ECU Composition
  - Application Components
    - CpSaDoorFrontLeft
    - CpSaInteriorLightFront
    - CpApMySwc
      - Port Prototypes
      - Application Connectors
      - Service Connectors
      - Task Mapping
      - Exclusive Area Implementation
      - Data Mapping
      - Memory Mapping
    - CpSaDoorFrontRight
    - CpOddIoHwAb
    - CpSaDisplay
  - Service Components
  - Service Mappings
  - Port Interface Mapping Sets

Triggered Function	Owner	Task	Position
RCtApMySwcCode	Component CpApMySwc		
RCtApMySwcInit	Component CpApMySwc	My_Task	0
RCtApMySwcComM_ModeChange_FULL_COMM_Exit	Component CpApMySwc	My_Task	2
RCtApMySwcComM_ModeChange_FULL_COMM_Entry	Component CpApMySwc	My_Task	3
RCtApMySwcPostRunCode	Component CpApMySwc	My_Task	4
		My_Task	4

Select Reference Target

Select Reference Target

Select one of the displayed containers as target for your reference.

- <NONE>
- IdleTask_OsCore
- Init_Task
- IO_Task
- My_Task
- SchM_Task

Create new reference target

Parent: Os

Manual Input

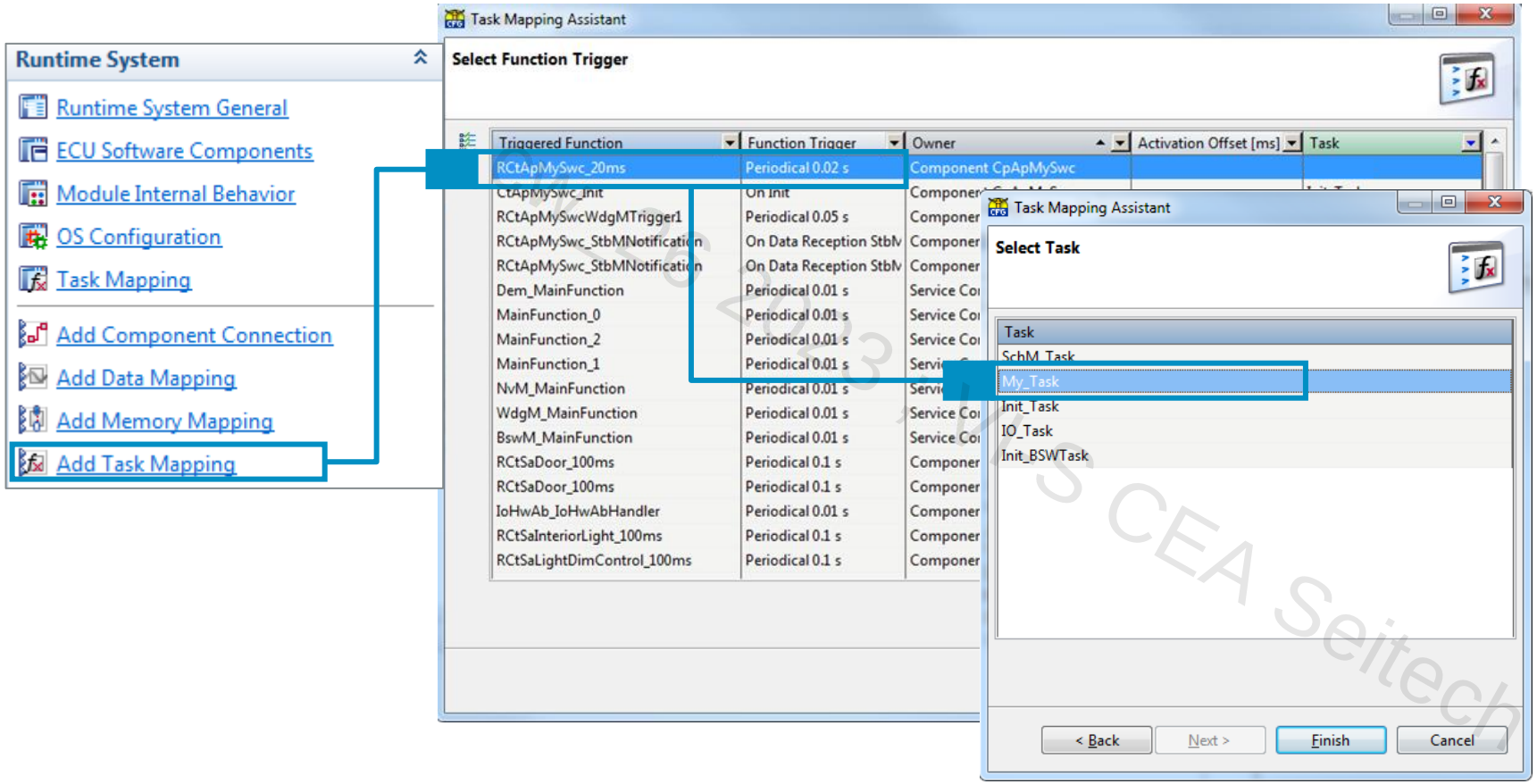
Reference Path:

OK Cancel

VECTOR 2023 , VI-S CEA Seitech

You can map Runnables to Tasks on a per-SWC approach.

Task mapping using the Task Mapping Assistant



Columns in the Task Mapping Assistant have auto-filter functionality. This makes it easy to map Runnables globally instead for just one SWC.

For example, you can map several Runnables in one step to a specific Task. The Assistant will choose the Position in the Task automatically.

Operating System

Software Components

cw_26 2023, VI-S CEA Seitech